

Jaka Mencinger

Backend Developer

Lesce, Slovenia · +386 51 328 817 · jack.mencinger@gmail.com

jakamencinger.dev · github.com/jaka2003 · linkedin.com/in/jaka-mencinger-0298b0401

PROFILE

Informatics graduate who has built and shipped two production SaaS products end to end: data model, API, deployment, backups and on-call. I work in TypeScript and Python. The engineering habits I lean on hardest: idempotent message handling, tolerance for unreliable networks, and integration tests against a real database rather than mocks. Day to day that means PostgreSQL, Docker, Linux deployment, CI/CD and REST APIs I designed and still maintain myself.

EXPERIENCE

Junior Software Engineer, Uroš Mencinger s.p. Oct 2025 – present

Independent development of SaaS products and web solutions, from requirements to production and maintenance.

- **Built and maintain** two SaaS products currently running in production, roughly 110,000 lines of my own code across 667 commits.
- I operate the production myself: deploys, database migrations, nightly backups to object storage, and email alerting on failure.
- Designed the relational schema for both products in PostgreSQL, including composite indexing and round-trip reduction work.
- Worked with clients from requirements through delivery; one food-service business uses the HACCP product daily and drives its roadmap.

Freelance web development Oct 2024 – Oct 2025

Shopify stores and websites for clients.

- Built and launched the Potepuh Shopify store: catalogue, collections, checkout, payments.
- Client websites in Next.js with a CMS the client manages independently.

Software Development Intern, Logix 3 months

- Worked inside a development team on React and Linux tasks, my first exposure to team process and code review.

Waiter, student work 3 years

- Communication under sustained pressure and shift-team coordination.

SELECTED ENGINEERING WORK

Kontrolnik: HACCP compliance SaaS kontrolnik.com · 2026

Next.js 16, TypeScript, Prisma 7, PostgreSQL, Stripe, Docker, Railway · 34 days from first commit to paying production

- **Offline-first sync.** Kitchen staff record temperatures on phones where wi-fi drops. I built a PWA with a hand-written service worker, an IndexedDB outbox and three sync endpoints. Every entry carries a client-generated ID that is unique in the database, so a replayed batch deduplicates instead of double-writing. Entries arriving more than 15 minutes late are flagged in reports, so an inspector can see the record was completed afterwards.
- **635 automated tests.** 232 unit, 377 integration and 26 end-to-end (~15,500 lines of test code). Integration tests run against a real PostgreSQL via embedded-postgres, no Docker needed; e2e runs on a mobile profile. Coverage includes concurrency races and immutability of closed records.
- **Multi-tenant isolation.** Every tenant check goes through a single entry point. The company ID is always derived from the location row in the database and never accepted from the client, which closes the obvious IDOR. Rejected reads return 404 rather than 403, so an attacker cannot confirm a record exists.
- **Upload hardening.** Layered: Content-Length pre-check, real size check, magic-byte verification against the declared type, then a sharp re-encode that strips EXIF/GPS and neutralises polyglot files. Stored filenames are always server-generated UUIDs.
- **Stripe webhooks.** Signature verified on the raw body. A manually granted plan was once overwritten by a renewal webhook that recomputed the tier from the price ID; I split manual grants into a separate column the webhook never writes, and shipped the migration with a comment recording the cause.

TvojAgent: real-estate CRM tvojagent.com · 2025–2026

FastAPI, Python, SQLAlchemy, Alembic, PostgreSQL, Next.js, Docker Compose, Cloudflare Tunnel · 456 commits, ~79,400 lines

- **Idempotent event handling.** Webhook delivery is at-least-once, so duplicates are inevitable. Processed events are keyed by the provider's event ID as primary key and ignored on redelivery; failed events are stored whole in a separate table, with admin endpoints to inspect and replay them without touching SQL.
- **Government registry integration.** Connected to the national cadastre (eProstor search API and OGC WFS for buildings, parcels and building parts) so a property can be created straight from public records. Coordinates convert from the Slovenian D96/TM grid (EPSG:3794) to WGS84 for map display, with pyproj server-side and proj4 in the browser. Public endpoints rate-limited to 10/min.
- **Round-trip reduction.** Dashboard endpoints cost roughly 15 round-trips per click. Added a 30-second TTL cache keyed by endpoint, company, user and role, with explicit invalidation at all seven write sites, plus nine composite indexes in a dedicated migration.
- **Multi-tenancy and privacy.** company_id foreign keys and indexes across ~15 tables with filtering at 120 call sites, all access routed through a single permissions module; archived records are immutable and return 409 even to admins.
- **Media pipeline.** Server-side ffmpeg transcoding of phone-recorded HEVC to H.264 behind an availability guard, plus WebP image variants, and generation never throws, so an upload still succeeds if resizing fails.

TECHNICAL SKILLS

Languages TypeScript, JavaScript, Python, SQL
Backend FastAPI, Node.js, Next.js route handlers, REST API design, zod / pydantic validation, Prisma, SQLAlchemy, Alembic
Data PostgreSQL: schema design, composite indexing, migrations, pg_dump backups; SQLite; IndexedDB
Infrastructure Docker, Linux/Ubuntu deployment, Cloudflare (Tunnel, R2), Railway, GitHub Actions, cron scheduling, nightly backups
Testing Vitest, Playwright, integration testing against real PostgreSQL, e2e on mobile device profiles
AI integration Anthropic API, structured outputs with schema validation, document OCR, cost and quota control
Coursework only C# / .NET: strongest language from my studies, but no production code in any repository

EDUCATION

Engineer of Informatics, Higher Vocational Diploma (EQF 6) *Kranj, Slovenia*
Višja strokovna šola Kranj, Informatics

LANGUAGES

- Slovenian: native · English: B2 · Serbo-Croatian: basic